Departamento de Computación
FCEFQyN, Universidad Nacional de Río Cuarto
Asignatura: Análisis y Diseño de Algoritmos I
Segundo Cuatrimestre de 2026

Guía Práctica No. 2: Fuerza Bruta y Búsqueda Exhaustiva

Esta guía práctica abarca una de las técnicas algorítmicas más básicas, Fuerza Bruta y Búsqueda Exhaustiva. Se recomienda fuertemente la lectura del capítulo 3 de *Introduction to the Design and Analysis of Algorithms* (Levitin 2003)

Parte de esta práctica cuenta con esquemas de programación y tests de asistencia a su resolución. Éstos pueden accederse a partir del classroom de github, para el cual es conveniente tener una cuenta en Github (`http://github.com`).

Esta práctica no tiene entrega formal. Su resolución es opcional aunque se recomienda realizar cada uno de los ejercicios propuestos.

1. Para los siguientes problemas, describa en palabras y/o pseudo-código (en papel), algoritmos que los resuelvan basándose en la técnica Fuerza Bruta/ Búsqueda Exhaustiva. Luego implemente, en Haskell o en Java, las soluciones descriptas.

   - Dado un conjunto s y un valor n, decidir si existe un subconjunto de s cuya suma sea n.
   - Decidir si un conjunto de enteros se puede particionar en dos conjuntos de igual suma.
   - Dadas dos cadenas, decidir si las mismas son anagramas (una es permutación de la otra).
   - Dadas dos cadenas p y s, decida si p es subcadena de s.
   - Dadas dos cadenas p y s, decida si p es subsecuencia de s (los elementos no necesariamente tienen que aparecer contiguos en s).
   - Dada una secuencia s de números, encontrar el k-ésimo elemento más grande en s.

2. Acceda al código de base del proyecto Haskell **listSorter**:

   - Implemente el algoritmo de ordenamiento *SlowSort*, que ordena una lista de elementos (por ejemplo, de enteros) mediante la generación de la lista de todas las permutaciones de la lista original, y filtrando aquella que está ordenada.
   - Agregue la definición del algoritmo de ordenamiento *BubbleSort*, que puede pensarse como una ligera mejora al algoritmo *SlowSort*, en la cual en lugar de generar todas las permutaciones, sólo se permutan items de la lista si éstos están desordenados.
   - Compare empíricamente *SlowSort* con *BubbleSort*, para evaluar las consecuencias de esta sencilla modificación.

3. Diseña e implementa un programa en Java o en Haskell que resuelva el problema de la mochila, mediante fuerza bruta (búsqueda exhaustiva). Argumente sobre la complejidad en el peor caso de su algoritmo.

4. Implemente en Java el algoritmo para resolver el problema de, dado un conjunto de puntos del plano, encontrar el par de puntos (distintos) cuya distancia es la menor entre todos los puntos del plano. Argumente sobre la complejidad de su solución en el peor caso.

5. Diseñe, e implemente en Java o Haskell, un algoritmo que elimine elementos repetidos de una lista. Argumente sobre la complejidad en peor caso de su algoritmo.

6. Considere la clase `CaesarCracker`, provista en el repositorio, y cuyo propósito es desencriptar, mediante fuerza bruta, un mensaje ASCII cifrado mediante un cifrado César.

   - Implemente el método `decode`, que dado un mensaje y la clave, desencripta el mismo.
   - Implemente el método `bruteForceDecrypt()`, que intenta desencriptar, por fuerza bruta, un mensaje encriptado, probando con todas las claves posibles de hasta tamaño k (véase atributo `passwordLength`) y utilizando como criterio de desencripción exitosa que el mensaje desencriptado contenga una palabra específica (véase atributo `messageWord`). Argumente sobre la complejidad de su solución.

7. Dado un conjunto de letras proposicionales $\{p1, p2, ..., pk\}$, se quiere construir el conjunto de todas las valuaciones posibles, es decir, todas las formas posibles de combinar las letras proposicionales y sus negaciones. Por ejemplo, si las letras proposicionales son $\{p, q\}$, entonces debe producirse como resultado el conjunto $\{[p, q], [\neg p, q], [p, \neg q], [\neg p, \neg q]\}$. Diseñe e implemente un algoritmo que resuelva este problema, usando Java o Haskell según su conveniencia.

8. Implemente en Java el problema de coloreo de grafos. El problema consiste en colorear todos los nodos de un grafo no dirigido, de manera tal que no existan en el grafo dos nodos adyacentes con el mismo color, usando la menor cantidad posible de colores.